

Laboratorio4_AntonioVinicius

September 24, 2026

1 LAB 4 - MÉTODOS NUMÉRICOS - ANTÔNIO VINÍCIUS LIBERATO OLIVEIRA SIEBRA - 610619

```
[1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP.models import (ModelSpec as MS,
                        summarize,
                        poly)
from sklearn.model_selection import train_test_split
```

Explicação: Importação das bibliotecas numéricas padrão, statsmodels para regressão, métodos de transformação do pacote ISLP, e a função train_test_split do scikit-learn para a criação manual do conjunto de validação. Observação: removemos o load_data pois usaremos o pandas para carregar o dataset em formato .csv.

```
[2]: from functools import partial
from sklearn.model_selection import \
    (cross_validate,
     KFold,
     ShuffleSplit)
from sklearn.base import clone
from ISLP.models import sklearn_sm
```

Explicação: Importações adicionais para as técnicas mais avançadas de Validação Cruzada do scikit-learn. O método sklearn_sm é um invólucro (wrapper) criado pelo ISLP que garante compatibilidade entre as APIs do statsmodels e do scikit-learn.

1.0.1 5.3.1 The Validation Set Approach

```
[3]: df_solar = pd.read_csv('dataset_regressao_limpo.csv')
df_solar.columns = df_solar.columns.str.strip()
# Reduzindo o dataset para 400 amostras para viabilizar métodos intensivos
# ↳ computacionalmente (como LOOCV)
df_solar = df_solar.sample(n=400, random_state=42).reset_index(drop=True)

df_train, df_valid = train_test_split(df_solar,
```

```
test_size=196,  
random_state=0)
```

Explicação: Carrega a base de dados de energia solar (`dataset_regressao_limpo.csv`), remove possíveis espaços nos nomes das colunas e sorteia 400 amostras aleatórias (para que métodos intensivos como o LOOCV não demorem muito). Em seguida, divide os dados aleatoriamente em treino e teste, extraindo 196 amostras para a base de validação. O `random_state=0` garante a reprodutibilidade dessa divisão.

```
[4]: irradi_mm = MS(['IRRADIATION'])  
X_train = irradi_mm.fit_transform(df_train)  
y_train = df_train['DC_POWER']  
model = sm.OLS(y_train, X_train)  
results = model.fit()
```

Explicação: Configura a matriz do modelo contendo exclusivamente a variável IRRADIATION (irradiação solar), ajusta essa matriz ao conjunto de treino recém-criado e treina um modelo de regressão linear simples para prever a geração de potência (DC_POWER).

```
[5]: X_valid = irradi_mm.transform(df_valid)  
y_valid = df_valid['DC_POWER']  
valid_pred = results.predict(X_valid)  
np.mean((y_valid - valid_pred)**2)
```

```
[5]: np.float64(212296.47014040715)
```

Explicação: Transforma o conjunto de validação sob as mesmas regras do conjunto de treino (focando na variável IRRADIATION), realiza a predição e calcula o Erro Quadrático Médio (MSE). Este valor de erro linear servirá de baseline de comparação.

```
[6]: def evalMSE(terms,  
               response,  
               train,  
               test):  
    mm = MS(terms)  
    X_train = mm.fit_transform(train)  
    y_train = train[response]  
    X_test = mm.transform(test)  
    y_test = test[response]  
    results = sm.OLS(y_train, X_train).fit()  
    test_pred = results.predict(X_test)  
    return np.mean((y_test - test_pred)**2)
```

Explicação: Para evitar repetição de código nas próximas validações polinomiais, encapsula-se toda a estrutura de ajuste e cálculo de Erro Quadrático Médio numa função utilitária chamada `evalMSE`.

```
[7]: MSE = np.zeros(3)  
for idx, degree in enumerate(range(1, 4)):
```

```

MSE[idx] = evalMSE([poly('IRRADIATION', degree)],
                    'DC_POWER',
                    df_train,
                    df_valid)
MSE

```

```
[7]: array([212296.47014041, 188101.63075968, 186673.3243211 ])
```

Explicação: Executa um loop avaliando modelos polinomiais utilizando a variável `IRRADIATION` nos graus 1 (linear), 2 (quadrático) e 3 (cúbico) para prever `DC_POWER`. Podemos analisar qual grau de polinômio reflete melhor a relação entre irradiação e potência gerada.

```

[8]: df_train, df_valid = train_test_split(df_solar,
                                          test_size=196,
                                          random_state=1)

MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('IRRADIATION', degree)],
                        'DC_POWER',
                        df_train,
                        df_valid)
MSE

```

```
[8]: array([187715.17407275, 164027.41091424, 174412.24442014])
```

Explicação: Repete o fluxo com uma nova divisão aleatória (`random_state=1`) para destacar a principal desvantagem da abordagem simples de Conjunto de Validação: os valores de erro flutuam severamente dependendo de quais amostras calharam no treino ou teste.

```

[9]: irrads_model = sklearn_sm(sm.OLS,
                               MS(['IRRADIATION']))
X, y = df_solar.drop(columns=['DC_POWER']), df_solar['DC_POWER']

validation = ShuffleSplit(n_splits=1, test_size=196, random_state=0)
results = cross_validate(irrads_model, X, y, cv=validation)
results['test_score']

```

```
[9]: array([212296.47014041])
```

Explicação: Trecho incluído nesta revisão. Demonstra a mesma técnica do *Validation Set Approach* vista acima, mas agora utilizando a ferramenta `ShuffleSplit` do `scikit-learn` de forma automática dentro do ecossistema de validação cruzada (`cross_validate`), dispensando todo o código manual.

```

[10]: validation = ShuffleSplit(n_splits=10, test_size=196, random_state=0)
results = cross_validate(irrads_model, X, y, cv=validation)
results['test_score'].mean(), results['test_score'].std()

```

```
[10]: (np.float64(196347.0161635367), np.float64(31643.483709556494))
```

Explicação: Trecho incluído nesta revisão. Comprova a alta variação na métrica usando 10 partições aleatórias (`n_splits=10`), extraindo a média do erro e o seu desvio padrão no final.

1.0.2 5.3.2 Cross-Validation (LOOCV e K-Fold)

```
[11]: cv_results = cross_validate(irrad_model,
                                X,
                                y,
                                cv=df_solar.shape[0])
cv_err = np.mean(cv_results['test_score'])
cv_err
```

```
[11]: np.float64(194766.2364355221)
```

Explicação: Aplica a Validação Cruzada *Leave-One-Out* (LOOCV). Ao configurar o número de folds (`cv`) idêntico ao número total de observações no nosso subset (`df_solar.shape[0]`), garante-se que o modelo será treinado $n - 1$ vezes, validando com a única amostra deixada de fora em cada iteração.

```
[12]: cv_error = np.zeros(5)
H = np.array(df_solar['IRRADIATION'])
M = sklearn_sm(sm.OLS)
for i, d in enumerate(range(1, 6)):
    X_poly = np.power.outer(H, np.arange(1, d + 1))
    M_CV = cross_validate(M,
                          X_poly,
                          y,
                          cv=df_solar.shape[0])
    cv_error[i] = np.mean(M_CV['test_score'])
cv_error
```

```
[12]: array([196827.70522283, 172085.72532211, 172046.31136929, 172431.76649629,
          171604.76733488])
```

Explicação: Automatiza o LOOCV para uma série de ajustes polinomiais (graus de 1 até 5). Usa o artifício matemático `np.power.outer` para gerar as matrizes com potências de forma ultra-rápida, contornando a construção repetitiva do `ModelSpec` a cada passo do loop.

```
[13]: cv_error = np.zeros(10)
cv = KFold(n_splits=10,
          shuffle=True,
          random_state=0)
for i, d in enumerate(range(1, 11)):
    X_poly = np.power.outer(H, np.arange(1, d + 1))
    M_CV = cross_validate(M,
                          X_poly,
                          y,
                          cv=cv)
    cv_error[i] = np.mean(M_CV['test_score'])
```

```
cv_error
```

```
[13]: array([194844.96155461, 170353.25389153, 169947.02576771, 170003.10329643,
          169206.96203822, 169481.25225097, 167413.61468993, 168391.22606765,
          170595.02065066, 176238.69495017])
```

Explicação: Executa a K-Fold Cross-Validation, método muito menos exigente computacionalmente que o LOOCV. Define-se $K=10$ *splits* e baralhamento (*shuffle*). O loop tenta polinômios da irradiação até ao grau 10, observando a estabilidade do erro conforme a complexidade do modelo aumenta.

1.0.3 5.3.3 The Bootstrap

```
[14]: def efficiency_func(D, idx):
      # Calcula uma métrica customizada: Rendimento/Eficiência Média
      # Razão entre a potência média gerada e a irradiação média recebida
      D_ = D.loc[idx]
      return D_['DC_POWER'].mean() / (D_['IRRADIATION'].mean() + 1e-8)
```

Explicação: Inicia os preparativos para o Bootstrap definindo uma métrica estatística própria. A função `efficiency_func` calcula a razão entre a potência média gerada (`DC_POWER`) e a irradiação média recebida (`IRRADIATION`) com base em um conjunto de amostras (índices), dando uma noção de rendimento (ou eficiência) do painel.

```
[15]: efficiency_func(df_solar, range(df_solar.shape[0]))
```

```
[15]: np.float64(13609.78118510769)
```

Explicação: Executa a função passando o índice de todas as amostras do nosso subset para retornar a eficiência pontual observada na base de dados inteira.

```
[16]: rng = np.random.default_rng(0)
      efficiency_func(df_solar,
                      rng.choice(df_solar.shape[0],
                                df_solar.shape[0],
                                replace=True))
```

```
[16]: np.float64(13715.42120938545)
```

Explicação: A base mecânica do Bootstrap: utiliza-se um gerador aleatório para sortear novos índices aleatórios *com reposição* (`replace=True`). Isso gera um “dataset falso” com o mesmo tamanho do original, e a métrica de eficiência é calculada novamente para essa nova amostragem.

```
[17]: def boot_SE(func,
                D,
                n=None,
                B=1000,
                seed=0):
      rng = np.random.default_rng(seed)
```

```

first_, second_ = 0, 0
n = n or D.shape[0]
for _ in range(B):
    idx = rng.choice(D.index,
                     n,
                     replace=True)

    value = func(D, idx)
    first_ += value
    second_ += value**2
return np.sqrt(second_ / B - (first_ / B)**2)

```

Explicação: Criação da função construtora do Bootstrap (`boot_SE`). Ela automatiza o loop por B iterações (por norma, 1000), gera as amostras com reposição, invoca a função alvo para reestimar a métrica sucessivamente, consolida os valores quadráticos e retorna o verdadeiro erro padrão (Standard Error).

```

[18]: eff_SE = boot_SE(eficiency_func,
                      df_solar,
                      B=1000,
                      seed=0)

eff_SE

```

```

[18]: np.float64(95.67464093872204)

```

Explicação: Enfim executa 1000 simulações de Bootstrap reais na nossa função de eficiência, gerando uma estimativa empírica e fidedigna do Desvio Padrão dessa métrica.

1.0.4 Estimando a Acurácia de um Modelo de Regressão Linear com Bootstrap

```

[19]: def boot_OLS(model_matrix, response, D, idx):
    D_ = D.loc[idx]
    Y_ = D_[response]
    X_ = clone(model_matrix).fit_transform(D_)
    return sm.OLS(Y_, X_).fit().params

```

Explicação: Como a regressão linear padrão já tenta assumir pressupostos teóricos em relação ao seu erro, criamos uma função Bootstrap exclusiva para ignorar esses pressupostos e forçar o recálculo dos coeficientes de regressão com índices amostrados na base.

```

[20]: irrads_func = partial(boot_OLS, MS(['IRRADIATION']), 'DC_POWER')

```

Explicação: Usa a poderosa ferramenta `partial` para pré-povoar os argumentos fixos (modelo linear e variável alvo `DC_POWER`) e deixar expostos apenas o dataframe e o índice, padrão necessário para a função `boot_SE` compreender o objeto.

```

[21]: irrads_func(df_solar, df_solar.index)

```

```

[21]: intercept      60.262815
      IRRADIATION    13346.133182

```

dtype: float64

Explicação: Verifica o retorno chamando o subproduto parcial usando como input os índices íntegros, imitando o modelo de ajuste original.

```
[22]: irradi_se = boot_SE(irrad_func,
                        df_solar,
                        B=1000,
                        seed=10)

irrad_se
```

```
[22]: intercept      12.610777
      IRRADIATION    119.482798
      dtype: float64
```

Explicação: Roda as 1000 iterações na regressão e exibe o Erro Padrão dos parâmetros com base na volatilidade empírica identificada nas observações criadas pelo Bootstrap.

```
[23]: irradi_model = sm.OLS(df_solar['DC_POWER'], MS(['IRRADIATION'])).
      ↪fit_transform(df_solar)).fit()
      summarize(irradi_model)
```

```
[23]:
```

	coef	std err	t	P> t
intercept	60.2628	28.228	2.135	0.033
IRRADIATION	13350.0000	77.737	171.683	0.000

Explicação: Ao usar a sumarização padrão do `statsmodels` e comparar os erros padrão analíticos com os erros estimados pelo Bootstrap acima (`irrad_se`), podemos constatar se há diferenças. Divergências significativas indicariam falhas em pressupostos teóricos (como linearidade ou homocedasticidade) quando usamos apenas a irradiação como preditor linear.

```
[24]: quad_model = MS([poly('IRRADIATION', 2, raw=True)])
      quad_func = partial(boot_OLS,
                        quad_model,
                        'DC_POWER')
      boot_SE(quad_func, df_solar, B=1000)
```

```
[24]: intercept      5.443817
      poly(IRRADIATION, degree=2, raw=True)[0]    271.069980
      poly(IRRADIATION, degree=2, raw=True)[1]    412.338818
      dtype: float64
```

Explicação: Substitui o modelo por uma versão quadrática polinomial, gerando novamente as reamostragens do Bootstrap para calcular os parâmetros (incluindo o termo `IRRADIATION^2`) sem premissas teóricas estritas.

```
[25]: quad_fit = sm.OLS(df_solar['DC_POWER'], quad_model.fit_transform(df_solar)).
      ↪fit()
      summarize(quad_fit)
```

[25] :

	coef	std err	t	P> t
intercept	-12.4081	28.192	-0.440	0.66
poly(IRRADIATION, degree=2, raw=True)[0]	14990.0000	232.154	64.586	0.00
poly(IRRADIATION, degree=2, raw=True)[1]	-2305.0963	308.365	-7.475	0.00

Explicação: *Trecho incluído nesta revisão.* A linha de desfecho do laboratório mostra o resumo estatístico do modelo quadrático nativo do `statsmodels`. Comparando com os erros encontrados no Bootstrap quadrático, avaliamos se este modelo representa mais fielmente a natureza dos dados solares, ajudando a mitigar eventuais inconsistências da regressão linear simples.